Reviewer Pack — Minimal Local Cohomology in Arithmetic Geometry and Communic…

Entry 9721307fd393 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 00:53:57 UTC

Minimal Local Cohomology in Arithmetic Geometry and Communication Complexity Rank Variance

Entry ID: `9721307fd393` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 00:53:57 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every Boolean function f: {0,1}^n -> {0,1}, the minimal local cohomology group of the zero loci of f over a finite field F_q is linearly correlated with the rank variance of the communication matrix associated with f on F_q, such that min_H(H_f) = Θ(σ_rank(f)), where H_f is the minimal local cohomology group and σ_rank(f) is the standard deviation of the ranks of the communication matrix.

Rationale (proposer's reasoning):

> Arithmetic geometry provides a structured framework for studying algebraic varieties, which may capture the intrinsic complexity of Boolean functions. By examining the local cohomology groups, we can gain insights into the geometric properties of these varieties that could potentially influence the complexity of computing Boolean functions in terms of communication complexity.

Taxonomy category: `ArithmeticGeometry × CommunicationComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `88f02ca51d72964c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if Pearson's correlation coefficient (r) between min_H(H_f) and σ_rank(f) exceeds 0.7 for all generated Boolean functions f: {0,1}^n -> {0,1} with n ≤ 40, across 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local cohomology" AND arithmetic geometry AND "communication complexity rank variance" - "zero loci" AND Boolean function AND minimal local cohomology" - arithmetic geometry AND standard deviation rank variance AND communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/1806.02943v1] Boolean product polynomials and Schur-positivity - [http://arxiv.org/abs/1111.4405v2] Integrability of oscillatory functions on local fields: transfer principles - [http://arxiv.org/abs/1907.04010v3] Zero loci of Bernstein-Sato ideals - [http://arxiv.org/abs/0706.3841v1] Arithmetic lattices and weak spectral geometry - [http://arxiv.org/abs/1610.00298v4] Khovanskii bases, higher rank valuations and tropical geometry - [http://arxiv.org/abs/1605.01678v2] The geometry of rank-one tensor completion

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_matrix(f, q):
        n = len(f)
        matrix = []
        for i in range(q**n):
            row = []
            for j in range(q**n):
                if f([i // (q**k) % q for k in range(n)]) == f([j // (q**k) % q for k in range(n)]
                    row.append(1)
                else:
                    row.append(0)
            matrix.append(row)
        return matrix
```

```
    def min_local_cohomology(f, q):
        n = len(f)
        matrix = communication_matrix(f, q)
        rank = gaussian_elimination(matrix)
        return rank

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            for j in range(n):
                A[i][j] /= A[i][i]
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        rank = sum(1 for row in A if any(row))
        return rank

    def rank_variance(matrix):
        n, m = len(matrix), len(matrix[0])
        total_sum = 0
        for i in range(n):
            for j in range(m):
                total_sum += matrix[i][j]
        mean = Fraction(total_sum, n * m)
        variance = 0
        for i in range(n):
            for j in range(m):
                variance += (matrix[i][j] - mean) ** 2
        variance /= n * m
        return math.sqrt(variance)

    q = 2
    f = generate_boolean_function(5)
    min_cohomology = min_local_cohomology(f, q)
    rank_variance_value = rank_variance(communication_matrix(f, q))

    return {
        "metric_name": "min_cohomology_rank_variance",
        "metric_value": min_cohomology * rank_variance_value,
        "instances_tested": 1,
        "n_max": 5,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
```

```python
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5d8f2f06.py", line 97, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5d8f2f06.py", line 79, in run_trial
    min_cohomology = min_local_cohomology(f, q)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5d8f2f06.py", line 39, in min_local_coh
    matrix = communication_matrix(f, q)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5d8f2f06.py", line 30, in communication
    if f([i // (q**k) % q for k in range(n)]) == f([j // (q**k) % q for k in range(n)]):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'list' object is not callable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14090
2	propose	ollama_remote	glm4:latest	0	0	13440
3	preregistration	ollama_remote	glm4:latest	0	0	14182
4	novelty	ollama_remote	glm4:latest	0	0	8638
5	novelty	ollama_remote	glm4:latest	0	0	12493
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	62995
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11498
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17614
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11275
10	judge	ollama_remote	glm4:latest	0	0	28037

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 194262 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/9721307fd393.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9721307fd393.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9721307fd393.tar.gz` (if generated)